OpenCV 기초

이미지 입출력 · 색공간

cchyun@gmail.com

Part 1. 컴퓨터 비전과 OpenCV

컴퓨터 비전이 로봇 개발에 왜 필요한가?

- **컴퓨터 비전은 로봇에게 "시각"이라는 핵심 감각을 부여하는 기술**

  - **카메라 없는 로봇 (Blind Robot)**
    - 시각 센서가 없어 미리 입력된 궤적만 반복하는 **"맹목적인 반복성"** 에 의존
    - 환경이 조금이라도 바뀌거나 장애물이 나타나면 적절히 대응하지 못하고 사고 발생

  - **카메라 있는 로봇 (Perception-driven Robot)**
    - 시각 시스템으로 주변 환경을 **인식(Perception)** 하여 지능적으로 행동
    - **물체 감지 및 좌표 판별**: 임의의 위치에 놓인 부품을 찾아 정확하게 집어 올림
    - **적응형 내비게이션**: 실시간 장애물 감지 및 경로 수정으로 자율 주행 수행
    - **인간-로봇 상호작용(HRI)**: 제스처·얼굴 인식으로 사람과 협업

OpenCV란 무엇인가? — 로봇 비전의 표준

- **OpenCV (Open Source Computer Vision Library)**
  - 컴퓨터 비전 및 머신러닝을 위한 강력한 오픈소스 라이브러리
  - 1999년 **Intel**에서 비전 기술 도입 가속화를 위해 최초 개발
- **로봇 비전의 표준이 된 이유**
  - **광범위한 기능**
    - 기초 이미지 필터링부터 객체 탐지, 얼굴 인식, 3D 재구성까지 방대한 알고리즘 포함
  - **다중 언어 및 플랫폼 지원**
    - C++, Python, Java / Windows, Linux, Android, macOS 등 거의 모든 환경 지원
  - **실시간 처리 능력**
    - 임베디드 시스템에서도 동작할 수 있도록 가볍고 빠르게 설계 — 로봇 공학에 최적
  - **강력한 커뮤니티**
    - 수많은 연구자·엔지니어가 기여 → 최신 논문 알고리즘이 빠르게 업데이트

Part 2. 설치 및 이미지 기본 구조

설치 가이드 — opencv-python vs opencv-contrib-python

- **파이썬 환경에서 OpenCV를 설치할 때 두 가지 옵션 중 하나를 선택**

  - `pip install opencv-python==4.10.0.84`
    - OpenCV의 **메인 모듈**만 포함
    - 이미지 읽기/쓰기, 필터링, 기하학적 변환 등 일반적인 비전 작업에 적합

  - `pip install opencv-contrib-python==4.10.0.84`
    - 메인 모듈 + **추가 모듈(extra modules)** 포함
    - SIFT, SURF 등 고급 특징점 알고리즘, 전문적인 얼굴 인식 알고리즘 포함
    - 로봇 비전의 고도화된 기능이 필요할 때 설치

  - **선택 기준**
    - 입문 단계: `opencv-python` 으로 충분
    - 고급 특징점 매칭·전문 기능: `opencv-contrib-python` 권장
    - ⚠️ 두 패키지를 동시에 설치하면 **충돌 발생** → 하나만 설치

이미지가 컴퓨터에서 표현되는 방식

- **컴퓨터는 이미지를 숫자로 이루어진 거대한 표, 즉 배열(Array)로 인식**
  - **픽셀 (Pixel)**
    - 이미지의 최소 단위인 점
    - 각 점에는 색상 정보가 숫자로 담겨 있음
  - **채널 (Channel) — 색상을 표현하는 성분**
    - **그레이스케일(Grayscale):** 밝기 정보만 있는 **1개의 채널** (0: 검은색 ~ 255: 흰색)
    - **컬러(Color):** 보통 빨강, 초록, 파랑의 조합으로 **3개의 채널**
  - **NumPy 배열과의 관계**
    - OpenCV에서 이미지는 파이썬의 **NumPy 다차원 배열** 구조로 저장
    - 배열의 크기: **(높이, 너비, 채널 수)** 순서로 표현

BGR 컨벤션 — OpenCV의 색상 채널 순서

- **중요: OpenCV는 기본적으로 BGR(파랑, 초록, 빨강) 순서를 사용**
  - 대부분의 프로그램(Matplotlib 등)은 **RGB** 순서 사용
  - OpenCV는 개발 당시 카메라 제조사 관행에 따라 **BGR** 채널 순서 채택

라이브러리	채널 순서	픽셀 배열 첫 번째 값
OpenCV	**B - G - R**	파랑(Blue)
Matplotlib, PIL 등	R - G - B	빨강(Red)

  - ⚠️ 이미지를 다룰 때 이 순서를 헷갈리면 색상이 이상하게 보임
  - 해결: `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` 로 변환

Part 3. 이미지 입출력

이미지 입출력 핵심 함수 4가지

- **로봇 카메라 이미지를 읽고, 보고, 저장하는 기본 흐름**

  - `cv2.imread(filepath)` — 이미지 파일 읽기
    - 파일을 읽어 NumPy 배열로 반환
    - 파일이 없으면 `None` 반환 (에러가 아님!) → 반드시 검사 필요

  - `cv2.imshow(window_name, image)` — 이미지 화면 출력
    - 지정한 이름의 윈도우 창에 이미지 표시

  - `cv2.waitKey(delay_ms)` — 키보드 입력 대기
    - `0`: 키를 누를 때까지 무한 대기 / 양수: 밀리초 단위 대기
    - 이 함수 없이 `imshow` 를 쓰면 창이 바로 닫힘

  - `cv2.imwrite(filepath, image)` — 이미지 파일 저장

(예제) 로봇 시야 확인하기

```python
import cv2  # OpenCV 라이브러리 임포트

# 1. imread: 이미지 파일 읽기
# 'robot_view.jpg' 파일을 읽어 NumPy 배열로 반환
# 파일이 없으면 None을 반환하므로 주의
image = cv2.imread('robot_view.jpg')

if image is not None:
    # 2. imshow: 이미지 화면 출력
    # 'Robot Eye View'라는 제목의 윈도우 창에 이미지 표시
    cv2.imshow('Robot Eye View', image)

    # 3. waitKey: 키보드 입력 대기
    # 0은 사용자가 키를 누를 때까지 무한 대기
    cv2.waitKey(0)

    # 4. imwrite: 이미지 파일 저장
    cv2.imwrite('processed_view.png', image)

    # 모든 창을 닫고 프로그램 종료
    cv2.destroyAllWindows()
else:
    print("이미지 파일을 찾을 수 없습니다.")
```

입문자가 자주 겪는 문제와 해결법

- **처음 OpenCV를 사용할 때 당황하기 쉬운 포인트** (1/2)

  - **① 이미지가 안 떠요 (NoneType Error)**
    - 원인: `cv2.imread()` 결과가 `None` → 파일 경로가 틀리거나 확장자가 다를 때 발생
    - 해결: 파일이 소스 코드와 같은 폴더에 있는지 확인 + `if image is not None:` 검사

  - **② 창이 응답 없음 상태가 돼요**
    - 원인: `cv2.imshow()` 다음에 `cv2.waitKey()` 호출을 빠뜨린 경우
    - 해결: 반드시 `cv2.waitKey()` 호출 — 윈도우 시스템 이벤트를 처리해줌

  - **③ 색상이 이상해요 (파란색 사람?)**
    - 원인: OpenCV(BGR) 이미지를 Matplotlib(RGB) 으로 바로 출력한 경우
    - 해결: `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` 로 변환 후 출력

입문자가 자주 겪는 문제와 해결법

- **처음 OpenCV를 사용할 때 당황하기 쉬운 포인트** (2/2)

  - **④ Google Colab에서 창이 안 열려요**
    - 원인: 클라우드 환경에서는 `cv2.imshow()` 직접 사용 불가
    - 해결: `from google.colab.patches import cv2_imshow` → `cv2_imshow(image)` 사용

Part 4. 색공간 변환

색공간(Color Space)이란?

- **색공간은 이미지에서 색상을 표현하기 위해 사용하는 수학적 규칙**

  - 우리가 물감을 섞어 색을 만들 때 빨강·파랑·노랑을 조합하듯
  - 컴퓨터도 여러 **채널(Channel)** 을 조합하여 하나의 색을 결정

  - **왜 여러 색공간이 필요한가?**
    - 각 색공간은 이미지의 **특정 속성** 을 더 잘 표현하도록 설계됨
    - 상황에 맞는 색공간을 선택하면 **연산 효율과 인식 정확도** 가 크게 향상

색공간	채널 구성	주요 활용
BGR	파랑·초록·빨강	기본 이미지 읽기·출력
Grayscale	밝기(1채널)	엣지 검출, 연산 속도 향상
HSV	색상·채도·명도	특정 색상 물체 탐지·추적
Lab	밝기·a·b	정밀한 색차 계산

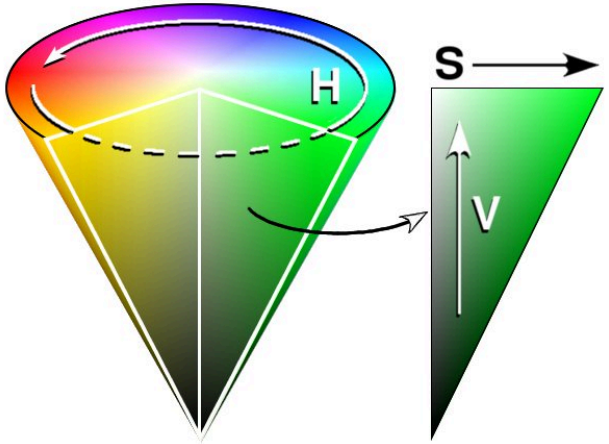
BGR vs RGB — 흔한 실수의 원인

- **BGR: OpenCV 기본 / RGB: 대부분의 라이브러리 표준**
  - **차이점**
    - 픽셀 데이터 배열의 채널 순서만 다름
    - BGR: 첫 번째 값 = 파란색(B), 마지막 값 = 빨간색(R)
    - RGB: 첫 번째 값 = 빨간색(R), 마지막 값 = 파란색(B)
  - **이유 (역사적 배경)**
    - OpenCV 초기 개발 당시 카메라 제조사·소프트웨어 공급업체 사이에서 BGR 포맷이 통용
  - **흔한 실수**
    - OpenCV(BGR)로 읽은 이미지를 **Matplotlib**(RGB 기대)으로 바로 출력
    - → **빨간색과 파란색이 서로 바뀌어** 색상이 어색하게 표시됨
    - 해결: `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` 로 변환 필수

주요 색공간 — Grayscale & HSV

- **Grayscale (회색조)**
  - 색상 정보를 없애고 0(검정) ~ 255(흰색)의 **밝기 정보** 만 남긴 상태
  - 채널 수가 3 → 1로 줄어들어 **연산 속도 향상**
  - 엣지 검출(Edge Detection), 이진화(Thresholding) 등 전처리에 필수 사용

- **HSV (색상·채도·명도)**
  - **Hue(색상):** 색의 종류 (빨강/초록/파랑 등의 색 자체)
  - **Saturation(채도):** 색의 선명도 (선명함 ↔ 회색)
  - **Value(명도):** 밝기 (밝음 ↔ 어두움)



  - **로봇 개발에서 HSV가 선호되는 이유**
    - 조명이 변해도 **'Hue(색상)'** 값은 비교적 안정적으로 유지
    - BGR은 조명 변화에 따라 색상 수치가 크게 달라져 물체 추적이 어려움
    - HSV는 특정 색상 물체 탐지·추적에 가장 효과적

cv2.cvtColor() — 색공간 변환 함수

- `cv2.cvtColor(src, code)`
  - 이미지의 색공간을 변환하는 함수

변환 코드	변환 방향	주요 사용 목적
`cv2.COLOR_BGR2GRAY`	BGR → Grayscale	엣지 검출, 연산 속도 향상
`cv2.COLOR_BGR2HSV`	BGR → HSV	특정 색상 물체 탐지·추적
`cv2.COLOR_BGR2RGB`	BGR → RGB	Matplotlib 출력용

```python
import cv2

image = cv2.imread('robot_view.jpg')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)   # Grayscale 변환
hsv  = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)    # HSV 변환
rgb  = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)    # Matplotlib 출력용
```

색공간 변환 후 채널 수 변화 주의사항

- **색공간 변환 시 배열의 차원(shape)이 달라짐**

```
import cv2

img = cv2.imread('robot_view.jpg')    # 640x480 컬러 이미지

print(img.shape)                                    # (480, 640, 3)  ← 3채널

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape)                                   # (480, 640)     ← 1채널 (2차원!)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
print(hsv.shape)                                    # (480, 640, 3)  ← 3채널 유지
```

- **BGR / RGB / HSV:** 높이 × 너비 × **3채널** (3차원 배열)
- **Grayscale:** 높이 × 너비로 구성된 **1채널 (2차원 배열)**

- ⚠️ 3채널 이미지를 기대하는 함수에 Grayscale 이미지를 넣으면 **에러 발생**
- → 처리 과정에서 데이터의 차원(`shape`)을 항상 확인하는 습관 중요

색공간 변환 — 입문자가 자주 하는 실수

- **처음 색공간을 다룰 때 주의해야 할 포인트**

  - **① 변환 없이 출력**
    - BGR 이미지를 RGB 표준 라이브러리(Matplotlib 등)로 바로 출력
    - → 색상이 왜곡되어 파란색 사람, 빨간 하늘 등 이상한 결과 출력

  - **② 컬러 이미지에 바로 알고리즘 적용**
    - 엣지 검출이나 이진화(Thresholding) 수행 전 Grayscale 변환을 빠뜨림
    - → 불필요하게 연산량이 늘어나거나 오류 발생

  - **③ HSV 범위 설정 오류**
    - 특정 색상을 찾을 때 Hue(색상) 범위를 잘못 설정하여 물체를 놓침
    - 예: 빨간색은 Hue 값이 **0 근처와 180 근처 두 군데** 에 존재하는 특이 케이스

(예제) HSV를 이용한 특정 색상(빨간색) 물체 탐지

```python
import cv2
import numpy as np

# 1. 이미지 로드 (BGR 포맷)
image = cv2.imread('obstacle.jpg')

# 2. HSV 색공간으로 변환
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# 3. 탐지할 빨간색의 HSV 범위 설정
# 주의: 빨간색은 Hue 값이 0 근처와 180 근처 두 군데에 존재
lower_red = np.array([0,   120, 70])   # Hue=0 근처 하한
upper_red = np.array([10, 255, 255])   # Hue=0 근처 상한

# 4. 범위에 해당하는 픽셀만 흰색, 나머지는 검정으로 마스크 생성
mask = cv2.inRange(hsv_image, lower_red, upper_red)

# 5. 원본 이미지에서 마스크에 해당하는 부분만 추출 (비트 연산)
detected_result = cv2.bitwise_and(image, image, mask=mask)

cv2.imshow('Original',           image)
cv2.imshow('Red Obstacle Mask',  mask)
cv2.imshow('Detected Result',    detected_result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

실습 체크리스트 — 로봇 카메라 시스템 구축

- **오늘 배운 내용을 실제로 구현했는지 확인하세요**

1. [ ] **환경 구축**
   `pip install opencv-python` 명령어로 라이브러리를 정상적으로 설치했는가?

2. [ ] **입력 확인**
   `cv2.VideoCapture()` 를 사용해 카메라에서 실시간 프레임을 정상적으로 받아올 수 있는가?

3. [ ] **출력 테스트**
   `cv2.imshow()` 로 프레임을 출력하고, 키보드 'q' 입력 시 안전하게 창이 닫히도록 설정했는가?

4. [ ] **색공간 활용**
   BGR 이미지를 HSV로 변환하고 `cv2.inRange()` 로 특정 색상을 마스킹해 보았는가?

5. [ ] **데이터 저장**
   `cv2.imwrite()` 를 사용하여 처리된 이미지를 `.jpg` 또는 `.png` 파일로 저장할 수 있는가?